

Seminarie 4: Strukturer och I/O

Carl Johan Ljungstedt Linda Bromander Ulf Borgenstam

6 November 2002

Strukturer och IO

Strukturer ger en möjlighet att gruppera variabler och behandla dem som en enhet.

Språket C innehåller inga I/O funktioner i sig själv utan förlitar sig på ett rikligt standardbibliotek. Biblioteket innehåller en mängd funktioner för in och utmatning till filer och strömmar.

1 Strukturer

1.1 Vad är en "struct" ?

Det är en samling av variabler som kan ha olika typ i motsats till arrayer som bara kan innehålla variabler av samma typ. För att deklarera en struct(hädanefter struktur) skriver man:

```
struct _Person {  
    char *namn;  
    int kod;  
    int niva;  
};
```

För att skapa en variabel av typen _Person skriver man:
struct _Person <variabelnamn>;

exempel:

```
struct _Person p1 = {  
    "Kalle_Bromander",  
    492,  
    9  
};
```

Här har vi deklarerat och tilldelat variabeln p1. Notera likheten med arrayer för tilldelningen.

exempel:

```
struct _Person {  
    char *namn;
```

```

        int kod;
        int niva;
    } p2, p3;

```

Här har vi deklarerat två variabler direkt efter strukturen. För att komma åt enskilda element i en stuktur använder man . (punkt).

exempel:

Om vi vill sätta koden i p2 till 456 skriver vi:

```
p2.kod = 456;
```

1.2 Nästade strukturer

Ibland kan vi behöva lägga strukturer inuti strukturer. Om vi vill lägga in strukturen

```

struct {
    int langd;
    int kod;
};

```

i strukturen _Person kan vi skriva:

```

struct _Person {
    char *namn;
    struct { int langd; int kod; } kod;
    int niva;
};

```

Notera att vi inte behöver ange något namn på själva strukturen.

exempel:

Så här tilldelar vi värden till variabler av den nya typen:

```

struct _Person p1 = {
    "Kalle_Bromander",
    { 4, 4928 },
    9
};

```

1.3 Typdefinitioner

```
typedef struct _Person Person;
```

tillåter oss att härnäst skriva Person istället för struct _Person. Person fungerar nu som vilken annan typ som helst.

exempel:

```
Person p2 = {  
    "Ulf_Bromander",  
    { 3, 785 },  
    6  
};
```

1.4 Arrayer, funktioner och tilldelning

Vi kan skapa arrayer av strukturer, använda tilldelningsoperatoren på dem och funktioner kan dessutom ta hela strukturer som argument eller returtyp.

exempel:

```
Person personal[2];  
  
Person skapaPerson(char *namn, int niva) {  
    Person p;  
  
    p.namn = namn;  
    p.kod = nykod(niva);  
    p.niva = niva;  
}  
  
personal[0] = p1;  
personal[1] = skapaPerson("Linda_Borgenstam", 10);
```

1.5 Pekare till strukturer

En pekare till en struktur skrivs som:

```
struct _Person *forsta;
```

Om vi det finns en typdefinition Person kan vi skriva:

```
Person *forsta = &p1;
```

För att komma åt element i en struktur till vilken vi har en pekare skriver man:

```
(*forsta).namn = "Niklas_Ljungstedt";
```

eller

```
forsta->namn = "Kalle_Ljungstedt";
```

som är en förenklad form. Paranteserna i första varianten måste finnas eftersom punkten binder hårdare.

1.6 Rekursiva strukturer

Man kan använda pekare till att skapa rekursiva datatyper. Här är ett exempel på en enkel länkad lista:

```
struct _Person {
    char *namn;
    struct { int langd; int kod; } kod;
    int niva;
    struct _Person *nasta;
};
```

En rekursiv struktur får inte innehålla sig själv utan måste i så fall innehålla en pekare.

exempel:

```
typedef struct _Person Person;

Person *laggTillPerson(Person *lista, char *namn, int niva) {
    Person *temp = sistaIListan(lista);

    temp->nasta = (Person *)malloc(sizeof(Person));
    *temp->nasta = skapaPerson(namn, niva);
    return nasta;
}
```

1.7 Bit-field

Ibland vill man manipulera data på bitnivå. Då kan man göra på två sätt, antingen kan man använda sig av de bitvisa operatorerna och maska fram de bitar man är intresserad av(jobbigt) eller så kan man låta C göra jobbet(jippie). Då kan man använda sig av bitfields som är en struktur där man kan ange bredden på elementen i antal bitar. Dessa packas sedan ner i en eller flera int.

```
volatile struct {
    unsigned int in : 1;
    unsigned int inniva : 4;
    unsigned int inlagnd : 5;
    unsigned int inkod : 22;
} *dorren;
```

Här har vi använt bitfält för att representera informationen i en minnesmappad I/O-port som läser av informationen från ett antal kodlås.

example:

```
void kolladorren() {
    while(!dorren->in) {}
}
```

```

        if (tilltrade(dorren->inniva, dorren->inlagnd, dorren->inkod) {
            oppnadorren();
        }
        else {
            rodLampa();
        }
    }
}

```

1.8 Union

Union är som en struktur fast alla element mappas till samma minnesadress, d v s. om man ändrar ett värde ändras alla andra också. Storleken på en union är lika stor som det största elementet.

```

union Test {
    int i;
    char c[13];
} t;

```

exmpel:

```

// Antag att t.c innehåller: "hello world!"

printf(t.c);

t.i=0x20202020;          // 0x20 är mellanslag i ASCII

printf(t.c);

// ger utskriften:
// hello world!
//   o world!

```

2 IO

2.1 Öppna filer

```
FILE *fopen(char *filnamn, char *mode);
```

fopen öppnar en fil och returnerar en pekare till en filström. mode är en sträng där man anger hur en fil ska öppnas. "r" öppnar för läsning och "w" för skrivning av en textfil.

```
int *fclose(FILE *f);
```

fclose stänger filen och frigör minne.

exempel:

```
void lasafil(char *filnamn) {
    FILE *f;

    f = fopen(filnamn, "r");
    if (f != NULL) {
        laspersonalfranfil(f);
        fclose(f);
    }
    else {
        fprintf(stderr, "det gick inte");
    }
}

void skrivfil(char *filnamn) {
    FILE *f;

    f = fopen(filnamn, "r");
    if (f != NULL) {
        skrivpersonaltillfill(f);
        fclose(f);
    }
    else {
        fprintf(stderr, "det gick inte");
    }
}
```

2.2 Vad är fprintf?

```
int fprintf(FILE *f, char *format, ...);
```

fprintf är en funktion som tar en fil, en formatsträng och ev. ett antal ytterligare argument. Om formatsträngen innehåller ett procenttecken (%) och följs av ett av nedanstående alternativ

output - omvandlingstecken för fprintf:

d,i	int: decimal heltal
o	int: unsigned oktaval heltal
xX	int: unsigned hexadecimal heltal
u	int: unsigned decimal heltal
c	int: single character
s	char*
f	double: [-]m.d, d är given precision
e,E	double: [-]m.d e (+ -)xx, där given precision

kommer nästa argument till funktionen att läsas in och skrivas ut.

exempel:

```
void skrivpersonaltillfil(FILE *f) {
    Person *p = forsta;
```

```

        while (p != NULL) {
            fprintf(f, "%s %i %i %i\n", p->namn, p->kod.langd,
                p->kod.kod, p->niva);
            p = p->nasta;
        }
    }
}

```

2.3 Vad är fscanf?

```
int fscanf(FILE *f, char *format, ...);
```

fscanf läser in data från en sträng och använder format strängen för att tolka indatan. Argumenten efter format måste vara pekare till variabler av rätt typ. format tolkas enligt:

input - omvandlingstecken för fscanf:

d,i	decimal heltal
o	unsigned oktav heltal
x	hexadecimal heltal
u	unsigned decimal heltal
c	single character
s	char*
e, f, g	flyttal

exempel:

```

int laspersonalfranfil(FILE *f) {
    int kodlangd, kod, niva;
    char *namn = (char *)malloc(50);    // Allokera 50 tecken

    while (fscanf(f, "%s %i %i %i", namn, &kodlangd, &kod, &niva)==4)
    {
        Person *p = laggTillPerson(namn, niva);

        p->kod.langd = kodlangd;
        p->kod.kod = kod;
    }
}

```

2.4 Standardströmar

stdin

stdin är standard strömmen för indata (ofta tangentbordet).

stdout

stdout är standardströmmen för utdata.

stderr

stderr är standardströmmen för felmeddelanden och dyl.

2.5 Övrig I/O

```
int getc(FILE *f);  
int getchar(void);
```

getc läser in ett tecken från strömmen f och returnerar det eller EOF vid filslut. getchar gör samma sak som getc(stdin).

```
int putc(int c, FILE *f);  
int putchar(int c);
```

putc skriver ett tecken till strömmen f och returnerar tecknet eller EOF vid filslut. putchar(c) gör samma sak som putc(c, stdout);

```
int printf(char *format, ...);
```

printf fungerar som fprintf med stdout som första argument.

```
int scanf(char *format, ...);
```

scanf fungerar som fscanf med stdin som första argument.

stderr och exit

För att skriva ut ett felmeddelande till stderr skriver man

```
fprintf(stderr, "Error: duh!");
```

Om man vill avsluta programmet direkt kan man använda

```
exit(int e);
```

där e är noll om allt är okej eller ett värde som talar om vad som är fel.

2.6 Sträng I/O

```
char *fgets(char *buffer, int maxlangd, FILE *f);
```

fgets läser en rad från strömmen f och sparar den i buffer. Funktionen läser maximalt maxlangd - 1 tecken och returnerar buffer om det lyckades eller NULL annars.

```
int fputs(char *line, FILE *f);
```

fputs skriver strängen line som en rad i filen f och returnerar 0 om det lyckas och EOF annars.

Seminarie 4: Strukturer och I/O

Carl Johan Ljungstedt Linda Bromander Ulf Borgenstam

6 November 2002

Strukturer och IO

Strukturer ger en möjlighet att gruppera variabler och behandla dem som en enhet.

Språket C innehåller inga I/O funktioner i sig själv utan förlitar sig på ett rikligt standardbibliotek. Biblioteket innehåller en mängd funktioner för in och utmatning till filer och strömmar.

1 Strukturer

1.1 Vad är en "struct" ?

Det är en samling av variabler som kan ha olika typ i motsats till arrayer som bara kan innehålla variabler av samma typ. För att deklarera en struct(hädanefter struktur) skriver man:

```
struct _Person {  
    char *namn;  
    int kod;  
    int niva;  
};
```

För att skapa en variabel av typen _Person skriver man:
struct _Person <variabelnamn>;

exempel:

```
struct _Person p1 = {  
    "Kalle_Bromander",  
    492,  
    9  
};
```

Här har vi deklarerat och tilldelat variabeln p1. Notera likheten med arrayer för tilldelningen.

exempel:

```
struct _Person {  
    char *namn;
```

```

        int kod;
        int niva;
    } p2, p3;

```

Här har vi deklarerat två variabler direkt efter strukturen. För att komma åt enskilda element i en stuktur använder man . (punkt).

exempel:

Om vi vill sätta koden i p2 till 456 skriver vi:

```
p2.kod = 456;
```

1.2 Nästade strukturer

Ibland kan vi behöva lägga strukturer inuti strukturer. Om vi vill lägga in strukturen

```

struct {
    int langd;
    int kod;
};

```

i strukturen _Person kan vi skriva:

```

struct _Person {
    char *namn;
    struct { int langd; int kod; } kod;
    int niva;
};

```

Notera att vi inte behöver ange något namn på själva strukturen.

exempel:

Så här tilldelar vi värden till variabler av den nya typen:

```

struct _Person p1 = {
    "Kalle_Bromander",
    { 4, 4928 },
    9
};

```

1.3 Typdefinitioner

```
typedef struct _Person Person;
```

tillåter oss att härnäst skriva Person istället för struct _Person. Person fungerar nu som vilken annan typ som helst.

exempel:

```
Person p2 = {  
    "Ulf_Bromander",  
    { 3, 785 },  
    6  
};
```

1.4 Arrayer, funktioner och tilldelning

Vi kan skapa arrayer av strukturer, använda tilldelningsoperatoren på dem och funktioner kan dessutom ta hela strukturer som argument eller returtyp.

exempel:

```
Person personal[2];  
  
Person skapaPerson(char *namn, int niva) {  
    Person p;  
  
    p.namn = namn;  
    p.kod = nykod(niva);  
    p.niva = niva;  
}  
  
personal[0] = p1;  
personal[1] = skapaPerson("Linda_Borgenstam", 10);
```

1.5 Pekare till strukturer

En pekare till en struktur skrivs som:

```
struct _Person *forsta;
```

Om vi det finns en typdefinition Person kan vi skriva:

```
Person *forsta = &p1;
```

För att komma åt element i en struktur till vilken vi har en pekare skriver man:

```
(*forsta).namn = "Niklas_Ljungstedt";
```

eller

```
forsta->namn = "Kalle_Ljungstedt";
```

som är en förenklad form. Paranteserna i första varianten måste finnas eftersom punkten binder hårdare.

1.6 Rekursiva strukturer

Man kan använda pekare till att skapa rekursiva datatyper. Här är ett exempel på en enkel länkad lista:

```
struct _Person {
    char *namn;
    struct { int langd; int kod; } kod;
    int niva;
    struct _Person *nasta;
};
```

En rekursiv struktur får inte innehålla sig själv utan måste i så fall innehålla en pekare.

exempel:

```
typedef struct _Person Person;

Person *laggTillPerson(Person *lista, char *namn, int niva) {
    Person *temp = sistaIListan(lista);

    temp->nasta = (Person *)malloc(sizeof(Person));
    *temp->nasta = skapaPerson(namn, niva);
    return nasta;
}
```

1.7 Bit-field

Ibland vill man manipulera data på bitnivå. Då kan man göra på två sätt, antingen kan man använda sig av de bitvisa operatorerna och maska fram de bitar man är intresserad av(jobbigt) eller så kan man låta C göra jobbet(jippie). Då kan man använda sig av bitfields som är en struktur där man kan ange bredden på elementen i antal bitar. Dessa packas sedan ner i en eller flera int.

```
volatile struct {
    unsigned int in : 1;
    unsigned int inniva : 4;
    unsigned int inlagnd : 5;
    unsigned int inkod : 22;
} *dorren;
```

Här har vi använt bitfält för att representera informationen i en minnesmappad I/O-port som läser av informationen från ett antal kodlås.

example:

```
void kolladorren() {
    while(!dorren->in) {}
}
```

```

        if (tilltrade(dorren->inniva, dorren->inlagnd, dorren->inkod) {
            oppnadorren();
        }
        else {
            rodLampa();
        }
    }
}

```

1.8 Union

Union är som en struktur fast alla element mappas till samma minnesadress, d v s. om man ändrar ett värde ändras alla andra också. Storleken på en union är lika stor som det största elementet.

```

union Test {
    int i;
    char c[13];
} t;

```

exmpel:

```

// Antag att t.c innehåller: "hello world!"

printf(t.c);

t.i=0x20202020;          // 0x20 är mellanslag i ASCII

printf(t.c);

// ger utskriften:
// hello world!
//   o world!

```

2 IO

2.1 Öppna filer

```
FILE *fopen(char *filnamn, char *mode);
```

fopen öppnar en fil och returnerar en pekare till en filström. mode är en sträng där man anger hur en fil ska öppnas. "r" öppnar för läsning och "w" för skrivning av en textfil.

```
int *fclose(FILE *f);
```

fclose stänger filen och frigör minne.

exempel:

```
void lasafil(char *filnamn) {
    FILE *f;

    f = fopen(filnamn, "r");
    if (f != NULL) {
        laspersonalfranfil(f);
        fclose(f);
    }
    else {
        fprintf(stderr, "det gick inte");
    }
}

void skrivfil(char *filnamn) {
    FILE *f;

    f = fopen(filnamn, "r");
    if (f != NULL) {
        skrivpersonaltillfill(f);
        fclose(f);
    }
    else {
        fprintf(stderr, "det gick inte");
    }
}
```

2.2 Vad är fprintf?

```
int fprintf(FILE *f, char *format, ...);
```

fprintf är en funktion som tar en fil, en formatsträng och ev. ett antal ytterligare argument. Om formatsträngen innehåller ett procenttecken (%) och följs av ett av nedanstående alternativ

output - omvandlingstecken för fprintf:

d,i	int: decimal heltal
o	int: unsigned oktaval heltal
xX	int: unsigned hexadecimal heltal
u	int: unsigned decimal heltal
c	int: single character
s	char*
f	double: [-]m.d, d är given precision
e,E	double: [-]m.d e (+ -)xx, där given precision

kommer nästa argument till funktionen att läsas in och skrivas ut.

exempel:

```
void skrivpersonaltillfil(FILE *f) {
    Person *p = första;
```

```

        while (p != NULL) {
            fprintf(f, "%s %i %i %i\n", p->namn, p->kod.langd,
                p->kod.kod, p->niva);
            p = p->nasta;
        }
    }
}

```

2.3 Vad är fscanf?

```
int fscanf(FILE *f, char *format, ...);
```

fscanf läser in data från en sträng och använder format strängen för att tolka indatan. Argumenten efter format måste vara pekare till variabler av rätt typ. format tolkas enligt:

input - omvandlingstecken för fscanf:

d,i	decimal heltal
o	unsigned oktav heltal
x	hexadecimal heltal
u	unsigned decimal heltal
c	single character
s	char*
e, f, g	flyttal

exempel:

```

int laspersonalfranfil(FILE *f) {
    int kodlangd, kod, niva;
    char *namn = (char *)malloc(50);    // Allokera 50 tecken

    while (fscanf(f, "%s %i %i %i", namn, &kodlangd, &kod, &niva)==4)
    {
        Person *p = laggTillPerson(namn, niva);

        p->kod.langd = kodlangd;
        p->kod.kod = kod;
    }
}

```

2.4 Standardströmar

stdin

stdin är standard strömmen för indata (ofta tangentbordet).

stdout

stdout är standardströmmen för utdata.

stderr

stderr är standardströmmen för felmeddelanden och dyl.

2.5 Övrig I/O

```
int getc(FILE *f);  
int getchar(void);
```

getc läser in ett tecken från strömmen f och returnerar det eller EOF vid filslut. getchar gör samma sak som getc(stdin).

```
int putc(int c, FILE *f);  
int putchar(int c);
```

putc skriver ett tecken till strömmen f och returnerar tecknet eller EOF vid filslut. putchar(c) gör samma sak som putc(c, stdout);

```
int printf(char *format, ...);
```

printf fungerar som fprintf med stdout som första argument.

```
int scanf(char *format, ...);
```

scanf fungerar som fscanf med stdin som första argument.

stderr och exit

För att skriva ut ett felmeddelande till stderr skriver man

```
fprintf(stderr, "Error: duh!");
```

Om man vill avsluta programmet direkt kan man använda

```
exit(int e);
```

där e är noll om allt är okej eller ett värde som talar om vad som är fel.

2.6 Sträng I/O

```
char *fgets(char *buffer, int maxlangd, FILE *f);
```

fgets läser en rad från strömmen f och sparar den i buffer. Funktionen läser maximalt maxlangd - 1 tecken och returnerar buffer om det lyckades eller NULL annars.

```
int fputs(char *line, FILE *f);
```

fputs skriver strängen line som en rad i filen f och returnerar 0 om det lyckas och EOF annars.

Seminarie 4: Strukturer och I/O

Carl Johan Ljungstedt Linda Bromander Ulf Borgenstam

6 November 2002

Strukturer och IO

Strukturer ger en möjlighet att gruppera variabler och behandla dem som en enhet.

Språket C innehåller inga I/O funktioner i sig själv utan förlitar sig på ett rikligt standardbibliotek. Biblioteket innehåller en mängd funktioner för in och utmatning till filer och strömmar.

1 Strukturer

1.1 Vad är en "struct" ?

Det är en samling av variabler som kan ha olika typ i motsats till arrayer som bara kan innehålla variabler av samma typ. För att deklarera en struct(hädanefter struktur) skriver man:

```
struct _Person {  
    char *namn;  
    int kod;  
    int niva;  
};
```

För att skapa en variabel av typen _Person skriver man:
struct _Person <variabelnamn>;

exempel:

```
struct _Person p1 = {  
    "Kalle_Bromander",  
    492,  
    9  
};
```

Här har vi deklarerat och tilldelat variabeln p1. Notera likheten med arrayer för tilldelningen.

exempel:

```
struct _Person {  
    char *namn;
```

```

        int kod;
        int niva;
    } p2, p3;

```

Här har vi deklarerat två variabler direkt efter strukturen. För att komma åt enskilda element i en struktur använder man . (punkt).

exempel:

Om vi vill sätta koden i p2 till 456 skriver vi:

```
p2.kod = 456;
```

1.2 Nästade strukturer

Ibland kan vi behöva lägga strukturer inuti strukturer. Om vi vill lägga in strukturen

```

struct {
    int langd;
    int kod;
};

```

i strukturen _Person kan vi skriva:

```

struct _Person {
    char *namn;
    struct { int langd; int kod; } kod;
    int niva;
};

```

Notera att vi inte behöver ange något namn på själva strukturen.

exempel:

Så här tilldelar vi värden till variabler av den nya typen:

```

struct _Person p1 = {
    "Kalle_Bromander",
    { 4, 4928 },
    9
};

```

1.3 Typdefinitioner

```
typedef struct _Person Person;
```

tillåter oss att härnäst skriva Person istället för struct _Person. Person fungerar nu som vilken annan typ som helst.

exempel:

```
Person p2 = {  
    "Ulf_Bromander",  
    { 3, 785 },  
    6  
};
```

1.4 Arrayer, funktioner och tilldelning

Vi kan skapa arrayer av strukturer, använda tilldelningsoperatoren på dem och funktioner kan dessutom ta hela strukturer som argument eller returtyp.

exempel:

```
Person personal[2];  
  
Person skapaPerson(char *namn, int niva) {  
    Person p;  
  
    p.namn = namn;  
    p.kod = nykod(niva);  
    p.niva = niva;  
}  
  
personal[0] = p1;  
personal[1] = skapaPerson("Linda_Borgenstam", 10);
```

1.5 Pekare till strukturer

En pekare till en struktur skrivs som:

```
struct _Person *forsta;
```

Om vi det finns en typdefinition Person kan vi skriva:

```
Person *forsta = &p1;
```

För att komma åt element i en struktur till vilken vi har en pekare skriver man:

```
(*forsta).namn = "Niklas_Ljungstedt";
```

eller

```
forsta->namn = "Kalle_Ljungstedt";
```

som är en förenklad form. Paranteserna i första varianten måste finnas eftersom punkten binder hårdare.

1.6 Rekursiva strukturer

Man kan använda pekare till att skapa rekursiva datatyper. Här är ett exempel på en enkel länkad lista:

```
struct _Person {
    char *namn;
    struct { int langd; int kod; } kod;
    int niva;
    struct _Person *nasta;
};
```

En rekursiv struktur får inte innehålla sig själv utan måste i så fall innehålla en pekare.

exempel:

```
typedef struct _Person Person;

Person *laggTillPerson(Person *lista, char *namn, int niva) {
    Person *temp = sistaIListan(lista);

    temp->nasta = (Person *)malloc(sizeof(Person));
    *temp->nasta = skapaPerson(namn, niva);
    return nasta;
}
```

1.7 Bit-field

Ibland vill man manipulera data på bitnivå. Då kan man göra på två sätt, antingen kan man använda sig av de bitvisa operatorerna och maska fram de bitar man är intresserad av(jobbigt) eller så kan man låta C göra jobbet(jippie). Då kan man använda sig av bitfields som är en struktur där man kan ange bredden på elementen i antal bitar. Dessa packas sedan ner i en eller flera int.

```
volatile struct {
    unsigned int in : 1;
    unsigned int inniva : 4;
    unsigned int inlagnd : 5;
    unsigned int inkod : 22;
} *dorren;
```

Här har vi använt bitfält för att representera informationen i en minnesmappad I/O-port som läser av informationen från ett antal kodlås.

example:

```
void kolladorren() {
    while(!dorren->in) {}
}
```

```

        if (tilltrade(dorren->inniva, dorren->inlagnd, dorren->inkod) {
            oppnadorren();
        }
        else {
            rodLampa();
        }
    }
}

```

1.8 Union

Union är som en struktur fast alla element mappas till samma minnesadress, d v s. om man ändrar ett värde ändras alla andra också. Storleken på en union är lika stor som det största elementet.

```

union Test {
    int i;
    char c[13];
} t;

```

exmpel:

```

// Antag att t.c innehåller: "hello world!"

printf(t.c);

t.i=0x20202020;          // 0x20 är mellanslag i ASCII

printf(t.c);

// ger utskriften:
// hello world!
//   o world!

```

2 IO

2.1 Öppna filer

```
FILE *fopen(char *filnamn, char *mode);
```

fopen öppnar en fil och returnerar en pekare till en filström. mode är en sträng där man anger hur en fil ska öppnas. "r" öppnar för läsning och "w" för skrivning av en textfil.

```
int *fclose(FILE *f);
```

fclose stänger filen och frigör minne.

exempel:

```
void lasafil(char *filnamn) {
    FILE *f;

    f = fopen(filnamn, "r");
    if (f != NULL) {
        laspersonalfranfil(f);
        fclose(f);
    }
    else {
        fprintf(stderr, "det gick inte");
    }
}

void skrivfil(char *filnamn) {
    FILE *f;

    f = fopen(filnamn, "r");
    if (f != NULL) {
        skrivpersonaltillfill(f);
        fclose(f);
    }
    else {
        fprintf(stderr, "det gick inte");
    }
}
```

2.2 Vad är fprintf?

```
int fprintf(FILE *f, char *format, ...);
```

fprintf är en funktion som tar en fil, en formatsträng och ev. ett antal ytterligare argument. Om formatsträngen innehåller ett procenttecken (%) och följs av ett av nedanstående alternativ

output - omvandlingstecken för fprintf:

d,i	int: decimal heltal
o	int: unsigned oktav heltal
xX	int: unsigned hexadecimal heltal
u	int: unsigned decimal heltal
c	int: single character
s	char*
f	double: [-]m.d, d är given precision
e,E	double: [-]m.d e (+ -)xx, där given precision

kommer nästa argument till funktionen att läsas in och skrivas ut.

exempel:

```
void skrivpersonaltillfil(FILE *f) {
    Person *p = forsta;
```

```

        while (p != NULL) {
            fprintf(f, "%s %i %i %i\n", p->namn, p->kod.langd,
                p->kod.kod, p->niva);
            p = p->nasta;
        }
    }
}

```

2.3 Vad är fscanf?

```
int fscanf(FILE *f, char *format, ...);
```

fscanf läser in data från en sträng och använder format strängen för att tolka indatan. Argumenten efter format måste vara pekare till variabler av rätt typ. format tolkas enligt:

input - omvandlingstecken för fscanf:

d,i	decimal heltal
o	unsigned oktav heltal
x	hexadecimal heltal
u	unsigned decimal heltal
c	single character
s	char*
e, f, g	flyttal

exempel:

```

int laspersonalfranfil(FILE *f) {
    int kodlangd, kod, niva;
    char *namn = (char *)malloc(50);    // Allokera 50 tecken

    while (fscanf(f, "%s %i %i %i", namn, &kodlangd, &kod, &niva)==4)
    {
        Person *p = laggTillPerson(namn, niva);

        p->kod.langd = kodlangd;
        p->kod.kod = kod;
    }
}

```

2.4 Standardströmar

stdin

stdin är standard strömmen för indata (ofta tangentbordet).

stdout

stdout är standardströmmen för utdata.

stderr

stderr är standardströmmen för felmeddelanden och dyl.

2.5 Övrig I/O

```
int getc(FILE *f);  
int getchar(void);
```

getc läser in ett tecken från strömmen f och returnerar det eller EOF vid filslut. getchar gör samma sak som getc(stdin).

```
int putc(int c, FILE *f);  
int putchar(int c);
```

putc skriver ett tecken till strömmen f och returnerar tecknet eller EOF vid filslut. putchar(c) gör samma sak som putc(c, stdout);

```
int printf(char *format, ...);
```

printf fungerar som fprintf med stdout som första argument.

```
int scanf(char *format, ...);
```

scanf fungerar som fscanf med stdin som första argument.

stderr och exit

För att skriva ut ett felmeddelande till stderr skriver man

```
fprintf(stderr, "Error: duh!");
```

Om man vill avsluta programmet direkt kan man använda

```
exit(int e);
```

där e är noll om allt är okej eller ett värde som talar om vad som är fel.

2.6 Sträng I/O

```
char *fgets(char *buffer, int maxlangd, FILE *f);
```

fgets läser en rad från strömmen f och sparar den i buffer. Funktionen läser maximalt maxlangd - 1 tecken och returnerar buffer om det lyckades eller NULL annars.

```
int fputs(char *line, FILE *f);
```

fputs skriver strängen line som en rad i filen f och returnerar 0 om det lyckas och EOF annars.